

使用 lit-element 建構 Brick Viewer

來源：<https://codelabs.developers.google.com/codelabs/lit-brick-viewer?hl=zh-tw>

步驟數：7

在本程式碼研究室中，您將運用 lit-element 元素建構 Brick Viewer 網頁元件。

目錄

1. [1. 簡介](#)
2. [2. 定義自訂元素](#)
3. [3. 指定 LDraw 檔案](#)
4. [4. 使用 Three.js 設定場景](#)
5. [5. 顯示部分模型](#)
6. [6. 磚塊組合導覽](#)
7. [7. 結論](#)

步驟 1 · 約 1 分鐘

1. 簡介

網頁元件

網頁元件是一組網頁標準，可讓開發人員使用自訂元素擴充 HTML。在本程式碼研究室中，您將定義 `<brick-viewer>` 元素，該元素可顯示積木模型！

lit-element

為協助定義自訂元素 `<brick-viewer>`，我們將使用 lit-element。lit-element 是輕量型基礎類別，可為網頁元件標準新增一些語法糖。這有助於我們輕鬆開始使用自訂元素。

開始使用

我們將在線上 Stackblitz 環境中編寫程式碼，因此請在新視窗中開啟這個連結：

stackblitz.com/edit/brick-viewer

立即開始！

2. 定義自訂元素

類別定義

如要定義自訂元素，請建立擴充 `LitElement` 的類別，並以 `@customElement` 加以裝飾。`@customElement` 的引數會是自訂元素的名稱。

自訂元素名稱必須包含破折號。

在 `brick-viewer.ts` 中輸入：

```
@customElement('brick-viewer')
export class BrickViewer extends LitElement {
}
```

現在，`<brick-viewer></brick-viewer>` 元素已可在 HTML 中使用。但如果您嘗試執行，則不會算繪任何內容。讓我們一起解決這個問題！

轉譯方法

如要實作元件的檢視區塊，請定義名為 `render` 的方法。這個方法應傳回以 `html` 函式標記的範本常值。在標記的範本常值中放入任何想要的 HTML。使用 `<brick-viewer>` 時，系統會顯示這項資訊。

新增 `render` 方法：

```
export class BrickViewer extends LitElement {
  render() {
    return html`<div>Brick viewer</div>`;
  }
}
```

3. 指定 LDraw 檔案

定義屬性

如果 `<brick-viewer>` 的使用者可以透過屬性指定要顯示的積木模型，那就太棒了，例如：

```
<brick-viewer src="path/to/model.ldraw"></brick-viewer>
```

由於我們要建構 HTML 元素，因此可以運用宣告式 API 並定義來源屬性，就像 `<img>` 或 `<video>` 標記一樣。使用 `lit-element` 時，只要以 `@property` 裝飾類別屬性即可。`type` 選項可讓您指定 `lit-element` 如何剖析屬性，以做為 HTML 屬性使用。

定義 `src` 屬性和屬性：

```
export class BrickViewer extends LitElement {  
  @property({type: String})  
  src: string|null = null;  
}
```

`<brick-viewer>` 現在有 `src` 屬性，我們可以在 HTML 中設定！由於 `lit-element`，我們已可從 `BrickViewer` 類別中讀取其值。

顯示值

我們可以在 `render` 方法的範本常值中使用 `src` 屬性，顯示該屬性的值。使用 `${value}` 語法將值插入範本常值。

```
export class BrickViewer extends LitElement {  
  render() {  
    return html`<div>Brick model: ${this.src}</div>`;  
  }  
}
```

現在，我們會在視窗中看到 `<brick-viewer>` 元素中 `src` 屬性的值。請嘗試開啟瀏覽器的開發人員工具，然後手動變更 `src` 屬性。快試試看吧！

...您是否注意到元素中的文字會自動更新？`lit-element` 會觀察以 `@property` 裝飾的類別屬性，並為您重新算繪檢視畫面！`lit-element` 會負責繁重的工作，您不必操心。

想知道重新算繪的效率嗎？`lit-element` 只會更新範本中變更的部分，不會進行 DOM 差異比較。

步驟 4 · 約 5 分鐘

4. 使用 Three.js 設定場景

燈光、攝影機、算繪！

我們的自訂元素會使用 three.js 算繪 3D 磚塊模型。我們希望為每個 `<brick-viewer>` 元素執行個體執行一次某些動作，例如設定 three.js 場景、攝影機和照明。我們會將這些項目新增至 `BrickViewer` 類別的建構函式。我們會將部分物件保留為類別屬性，以便稍後使用：攝影機、場景、控制項和算繪器。

加入 three.js 場景設定：

```

export class BrickViewer extends LitElement {

  private _camera: THREE.PerspectiveCamera;
  private _scene: THREE.Scene;
  private _controls: OrbitControls;
  private _renderer: THREE.WebGLRenderer;

  constructor() {
    super();

    this._camera = new THREE.PerspectiveCamera(45, this.clientWidth/this.clientHeight, 1, 10000);
    this._camera.position.set(150, 200, 250);

    this._scene = new THREE.Scene();
    this._scene.background = new THREE.Color(0xdeebed);

    const ambientLight = new THREE.AmbientLight(0xdeebed, 0.4);
    this._scene.add( ambientLight );

    const directionalLight = new THREE.DirectionalLight(0xffffff, 1);
    directionalLight.position.set(-1000, 1200, 1500);
    this._scene.add(directionalLight);

    this._renderer = new THREE.WebGLRenderer({antialias: true});
    this._renderer.setPixelRatio(window.devicePixelRatio);
    this._renderer.setSize(this.offsetWidth, this.offsetHeight);

    this._controls = new OrbitControls(this._camera, this._renderer.domElement);
    this._controls.addEventListener("change", () =>
      requestAnimationFrame(this._animate)
    );

    this._animate();

    const resizeObserver = new ResizeObserver(this._onResize);
    resizeObserver.observe(this);
  }

  private _onResize = (entries: ResizeObserverEntry[]) => {
    const { width, height } = entries[0].contentRect;
    this._renderer.setSize(width, height);
    this._camera.aspect = width / height;
    this._camera.updateProjectionMatrix();
    requestAnimationFrame(this._animate);
  };

  private _animate = () => {
    this._renderer.render(this._scene, this._camera);
  };
}

```

`WebGLRenderer` 物件提供 DOM 元素，可顯示算繪的 `three.js` 場景。可透過 `domElement` 屬性存取。我們可以使用 `${value}` 語法，將這個值插入算繪範本常值。

移除範本中的 `src` 訊息，並插入算繪器的 DOM 元素：

```
export class BrickViewer extends LitElement {
  render() {
    return html`
      ${this._renderer.domElement}
    `;
  }
}
```

如要完整顯示轉譯器的 DOM 元素，我們也需要將 `<brick-viewer>` 元素本身設為 `display: block`。我們可以在名為 `styles` 的靜態屬性中提供樣式，並設為 `css` 範本常值。

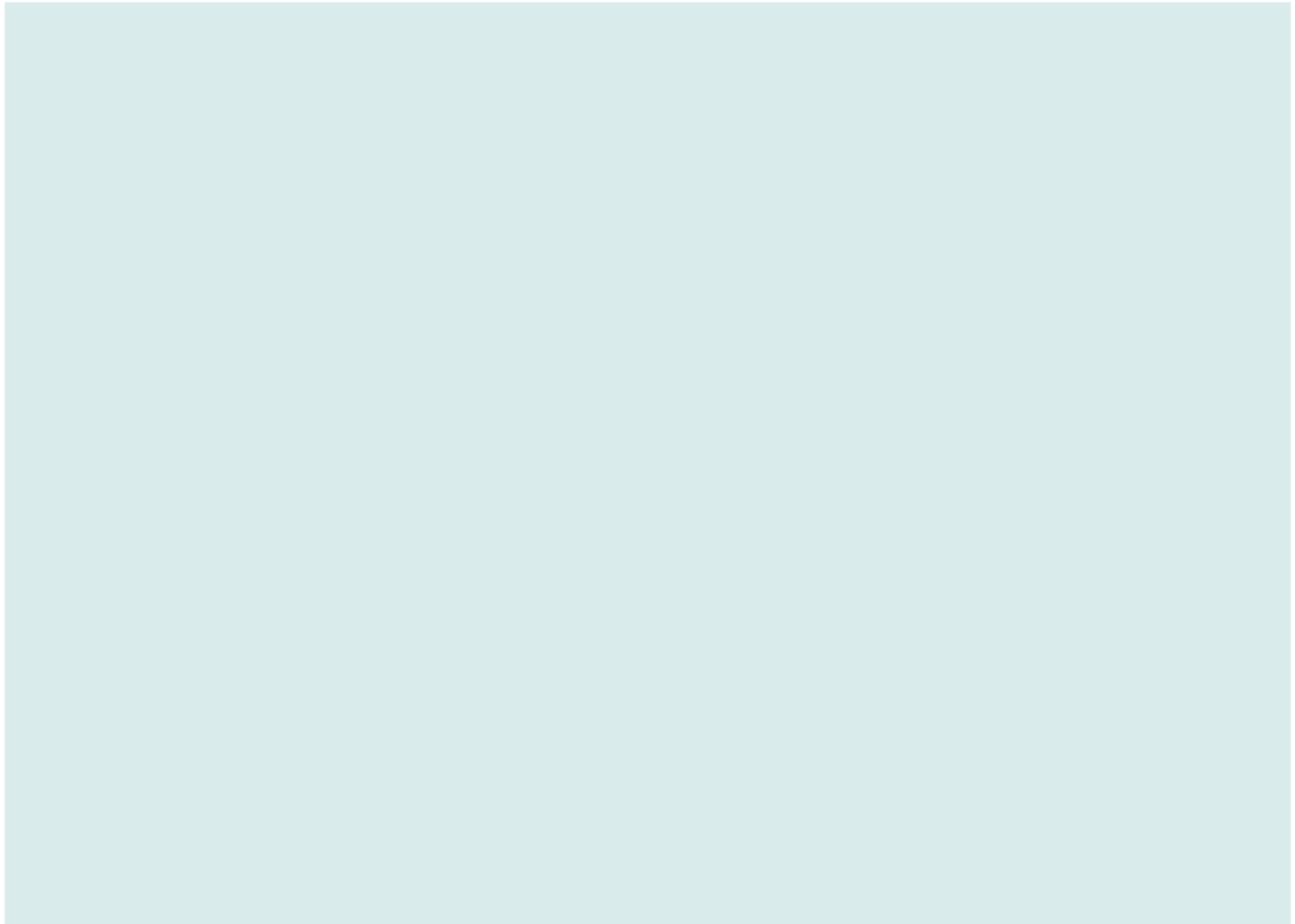
將這個樣式新增至類別：

```
export class BrickViewer extends LitElement {
  static styles = css`
    /* The :host selector styles the brick-viewer itself! */
    :host {
      display: block;
    }
  `;
}
```

現在 `<brick-viewer>` 會顯示算繪的 `three.js` 場景：

Brick Viewer

Car ▼



但...裡面是空的。我們來提供模型。

磚塊載入器

我們會將先前定義的 `src` 屬性傳遞至 `LDrawLoader`，該屬性會隨附於 `three.js`。

`LDraw` 檔案可將積木模型分成不同的建構步驟。您可以使用 `LDrawLoader` API 存取步驟總數和個別積木的可視度。

複製這些屬性、新的 `_loadModel` 方法，以及建構函式中的新行：

```

@customElement('brick-viewer')
export class BrickViewer extends LitElement {
  private _loader = new LDrawLoader();
  private _model: any;
  private _numConstructionSteps?: number;
  step?: number;

  constructor() {
    // ...
    // Add this line right before this._animate();
    (this._loader as any).separateObjects = true;
    this._animate();
  }

  private _loadModel() {
    if (this.src === null) {
      return;
    }
    this._loader
      .setPath('')
      // Using our src property!
      .load(this.src, (newModel) => {

        if (this._model !== undefined) {
          this._scene.remove(this._model);
          this._model = undefined;
        }

        this._model = newModel;

        // Convert from LDraw coordinates: rotate 180 degrees around OX
        this._model.rotation.x = Math.PI;
        this._scene.add(this._model);

        this._numConstructionSteps = this._model.userData.numConstructionSteps;
        this.step = this._numConstructionSteps!;

        const bbox = new THREE.Box3().setFromObject(this._model);
        this._controls.target.copy(bbox.getCenter(new THREE.Vector3()));
        this._controls.update();
        this._controls.saveState();
      });
  }
}

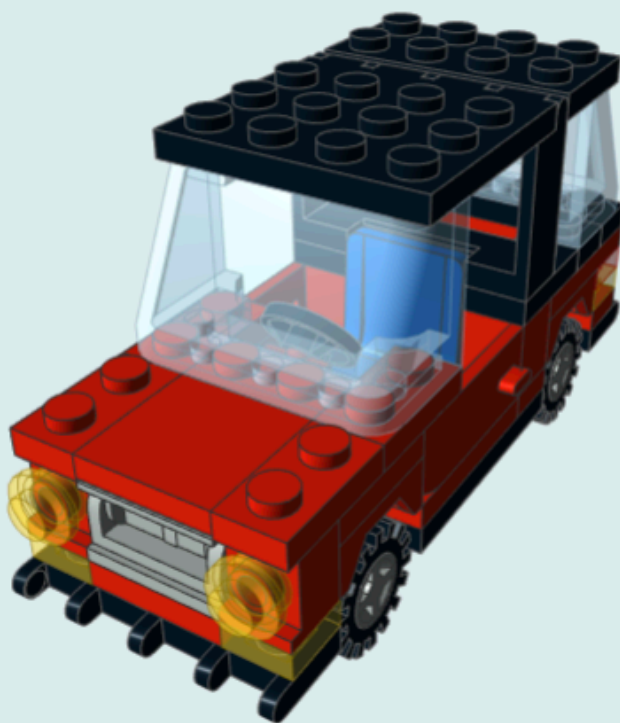
```

何時應呼叫 `_loadModel`？每次 `src` 屬性變更時，都必須呼叫這個函式。使用 `@property` 裝飾 `src` 屬性，即可選擇將屬性納入 `lit-element` 更新生命週期。每當其中一個裝飾屬性的值變更時，系統就會呼叫一系列方法，這些方法可以存取屬性的新舊值。我們感興趣的生命週期方法稱為 `update`。`update` 方法會採用 `PropertyValues` 引數，其中包含剛變更的任何屬性相關資訊。這裡非常適合呼叫 `_loadModel`。

新增 `update` 方法：

```
export class BrickViewer extends LitElement {  
  update(changedProperties: PropertyValues) {  
    if (changedProperties.has('src')) {  
      this._loadModel();  
    }  
    super.update(changedProperties);  
  }  
}
```

我們的 `<brick-viewer>` 元素現在可以顯示以 `src` 屬性指定的磚塊檔案。



瞭解詳情：如要瞭解 `lit-element` 更新生命週期，請前往 lit-element.polymer-project.org/guide/lifecycle。

步驟 5 · 約 3 分鐘

5. 顯示部分模型

現在，我們來設定目前的建構步驟。我們希望能夠指定 `<brick-viewer step="5"></brick-viewer>`，並查看第 5 個建構步驟的積木模型。如要這麼做，請使用 `@property` 裝飾器，將 `step` 屬性設為觀察屬性。

裝飾 `step` 屬性：

```
export class BrickViewer extends LitElement {
  @property({type: Number})
  step?: number;
}
```

現在，我們要新增輔助方法，只顯示目前建構步驟的積木。我們會在更新方法中呼叫輔助程式，以便在 `step` 屬性變更時執行。

更新 `update` 方法，並新增 `_updateBricksVisibility` 方法：

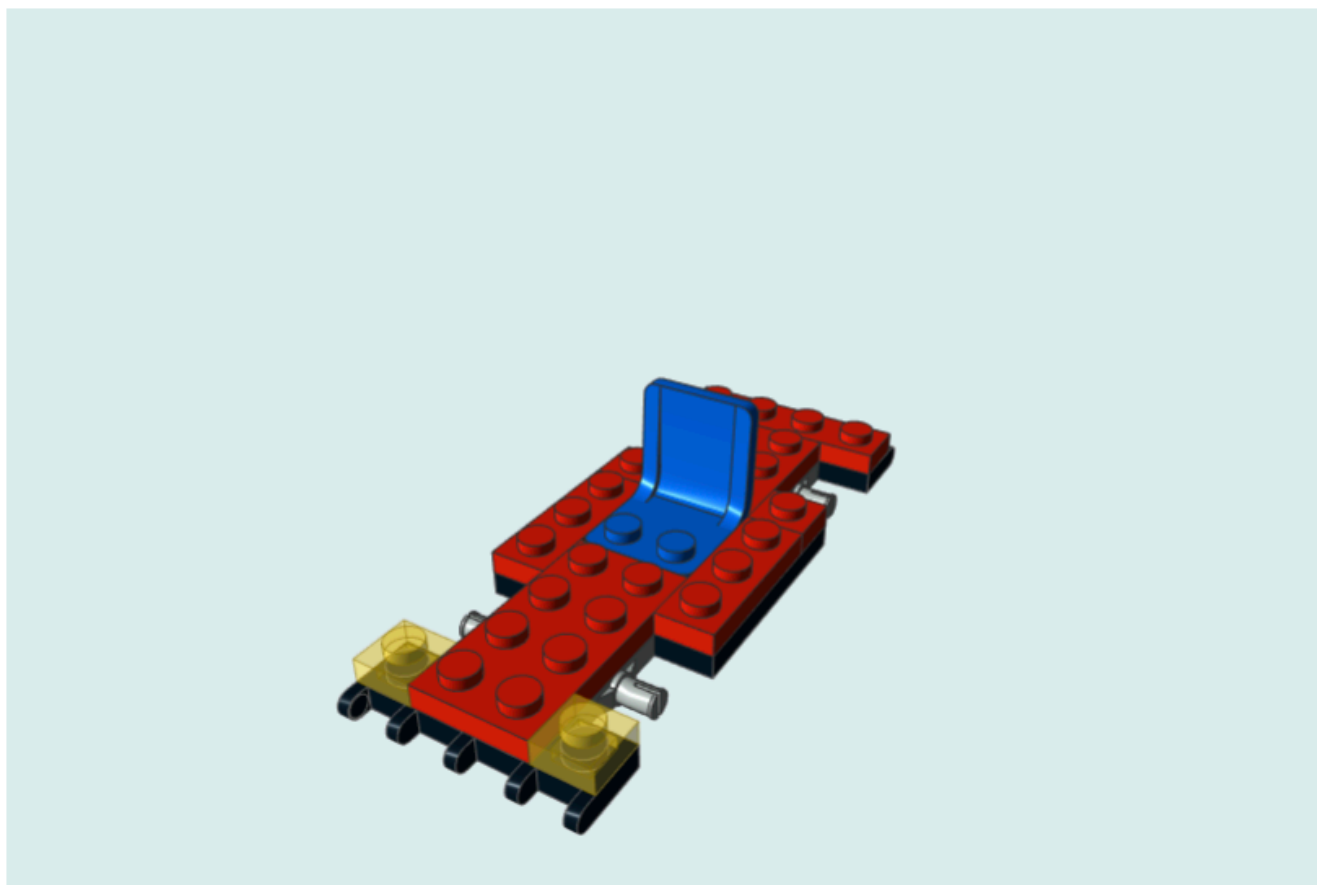
```
export class BrickViewer extends LitElement {
  update(changedProperties: PropertyValues) {
    if (changedProperties.has('src')) {
      this._loadModel();
    }
    if (changedProperties.has('step')) {
      this._updateBricksVisibility();
    }
    super.update(changedProperties);
  }

  private _updateBricksVisibility() {
    this._model && this._model.traverse((c: any) => {
      if (c.isGroup && this.step) {
        c.visible = c.userData.constructionStep <= this.step;
      }
    });
    requestAnimationFrame(this._animate);
  }
}
```

現在請開啟瀏覽器的開發人員工具，然後檢查 `<brick-viewer>` 元素。在其中加入 `step` 屬性，如下所示：



看看算繪模型會發生什麼變化！我們可以透過 `step` 屬性，控制模型顯示的範圍。將 `step` 屬性設為 "10" 時，畫面應如下所示：



步驟 6 · 約 8 分鐘

6. 磚塊組合導覽

mwc-icon-button

`<brick-viewer>` 的使用者也應能透過 UI 瀏覽建構步驟。讓我們新增按鈕，以便前往下一個步驟、上一個步驟和第一個步驟。我們會使用 Material Design 的按鈕網頁元件，讓這個過程更加輕鬆。由於 `@material/mwc-icon-button` 已匯入，我們現在可以放入 `<mwc-icon-button></mwc-icon-button>`。我們可以透過 `icon` 屬性指定要使用的圖示，如下所示：`<mwc-icon-button icon="thumb_up"></mwc-icon-button>`。如要查看所有可能的圖示，請前往 material.io/resources/icons。

我們在算繪方法中新增一些圖示按鈕：

```
export class BrickViewer extends LitElement {
  render() {
    return html`
      ${this._renderer.domElement}
      <div id="controls">
        <mwc-icon-button icon="replay"></mwc-icon-button>
        <mwc-icon-button icon="navigate_before"></mwc-icon-button>
        <mwc-icon-button icon="navigate_next"></mwc-icon-button>
      </div>
    `;
  }
}
```

有了網頁元件，在網頁上使用 Material Design 就是這麼簡單！

事件繫結

這些按鈕應實際執行某些動作。「回覆」按鈕應會將建構步驟重設為 1。「navigate_before」按鈕應遞減建構步驟，「navigate_next」按鈕則應遞增。lit-element 透過事件繫結，可輕鬆新增這項功能。在 HTML 範本常值中，使用 `@eventName=${eventHandler}` 語法做為元素屬性。現在，系統偵測到該元素上的 `eventName` 事件時，就會執行 `eventHandler`！舉例來說，我們為三個圖示按鈕新增點擊事件處理常式：

```

export class BrickViewer extends LitElement {
  private _restart() {
    this.step! = 1;
  }

  private _stepBack() {
    this.step! -= 1;
  }

  private _stepForward() {
    this.step! += 1;
  }

  render() {
    return html`
      ${this._renderer.domElement}
      <div id="controls">
        <mwc-icon-button @click=${this._restart} icon="replay"></mwc-icon-button>
        <mwc-icon-button @click=${this._stepBack} icon="navigate_before"></mwc-icon-button>
        <mwc-icon-button @click=${this._stepForward} icon="navigate_next"></mwc-icon-button>
      </div>
    `;
  }
}

```

請嘗試點選按鈕。做得好！

樣式

按鈕可以正常運作，但外觀不佳。他們全都擠在底部。現在來設定樣式，將這些元素疊加在場景上。

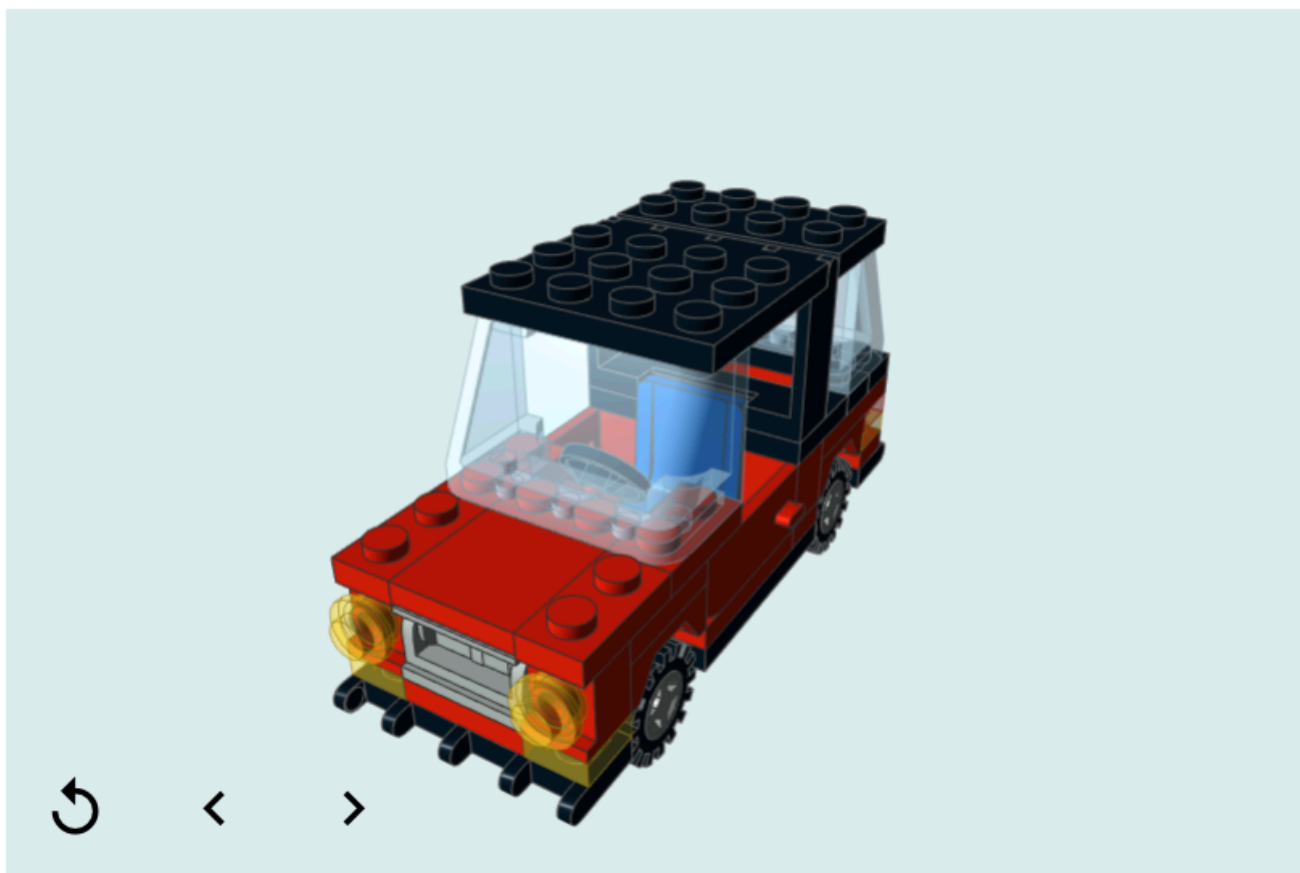
如要將樣式套用至這些按鈕，請返回 `static styles` 屬性。這些樣式會限定範圍，也就是只會套用至這個網頁元件內的元素。這也是撰寫網頁元件的樂趣之一：選取器可以更簡單，CSS 也會更容易讀取和撰寫。再見，[BEM](#)！

更新樣式，使其如下所示：

```

export class BrickViewer extends LitElement {
  static styles = css`
    :host {
      display: block;
      position: relative;
    }
    #controls {
      position: absolute;
      bottom: 0;
      width: 100%;
      display: flex;
    }
  `;
}

```



攝影機重設按鈕

`<brick-viewer>` 的使用者可以透過滑鼠控制項旋轉場景。新增按鈕時，我們也來新增一個按鈕，將攝影機重設為預設位置。另一個具有點擊事件繫結的 `<mwc-icon-button>` 即可完成這項工作。

```
export class BrickViewer extends LitElement {
  private _resetCamera() {
    this._controls.reset();
  }

  render() {
    return html`
      <div id="controls">
        <!-- ... -->
        <!-- Add this button: -->
        <mwc-icon-button @click=${this._resetCamera} icon="center_focus_strong"></mwc-icon-
button>
      </div>
    `;
  }
}
```

更便捷的帳戶導覽

有些積木組有很多步驟。使用者可能想略過特定步驟。新增含有步數的滑桿，有助於快速瀏覽。我們會使用 `<mwc-slider>` 元素。

mwc-slider

滑桿元素需要一些重要資料，例如滑桿的最小值和最大值。滑桿的最小值一律為「1」。如果模型已載入，滑桿最大值應為 `this._numConstructionSteps`。我們可以透過 `<mwc-slider>` 的屬性得知這些值。如果 `_numConstructionSteps` 屬性尚未定義，我們也可以使用 `ifDefined` `lit-html` 指令，避免設定 `max` 屬性。

在「back」和「forward」按鈕之間新增 `<mwc-slider>`：

```
export class BrickViewer extends LitElement {
  render() {
    return html`
      <div id="controls">
        <!-- ... backwards button -->
        <!-- Add this slider: -->
        <mwc-slider
          step="1"
          pin
          markers
          min="1"
          max=${ifDefined(this._numConstructionSteps)}
        ></mwc-slider>
        <!-- ... forwards button -->
      </div>
    `;
  }
}
```

資料「向上」

使用者移動滑桿時，目前的建構步驟應會變更，模型的顯示狀態也應隨之更新。每當拖曳滑桿時，滑桿元素就會發出輸入事件。在滑桿本身新增事件繫結，以擷取這個事件並變更建構步驟。

新增事件繫結：

```
export class BrickViewer extends LitElement {
  render() {
    return html`
      <div id="controls">
        <!-- ... -->
        <!-- Add the @input event binding: -->
        <mwc-slider
          ...
          @input=${(e: CustomEvent) => this.step = e.detail.value}
        ></mwc-slider>
        <!-- ... -->
      </div>
    `;
  }
}
```

太棒了！我們可以使用滑桿變更顯示的步驟。

資料「下降」

還有一件事。使用「上一步」和「下一步」按鈕變更步驟時，必須更新滑桿控點。將 `<mwc-slider>` 的值屬性繫結至 `this.step`。

使用網頁元件時，請透過事件從元素「向上」傳播資料，並透過屬性「向下」傳播資料至子項。

新增 `value` 繫結：

```
export class BrickViewer extends LitElement {
  render() {
    return html`
      <div id="controls">
        <!-- ... -->
        <!-- Add the value property binding: -->
        <mwc-slider
          ...
          value=${ifDefined(this.step)}
        ></mwc-slider>
        <!-- ... -->
      </div>
    `;
  }
}
```

我們即將完成滑桿。新增彈性樣式，讓其他控制項能順利運作：

```
export class BrickViewer extends LitElement {
  static styles = css`
    /* ... */
    mwc-slider {
      flex-grow: 1;
    }
  `;
}
```

此外，我們也需要在滑桿元素本身呼叫 `layout`。我們會在 `firstUpdated` 生命週期方法中執行這項操作，DOM 首次配置完成後，系統就會呼叫這個方法。`query` 裝飾器可協助我們取得範本中滑桿元素的參照。

```
export class BrickViewer extends LitElement {
  @query('mwc-slider')
  slider!: Slider|null;

  async firstUpdated() {
    if (this.slider) {
      await this.slider.updateComplete
      this.slider.layout();
    }
  }
}
```

以下是所有新增的滑桿 (滑桿上還有額外的 `pin` 和 `markers` 屬性，讓滑桿看起來更酷)：

```

export class BrickViewer extends LitElement {
  @query('mwc-slider')
  slider!: Slider|null;

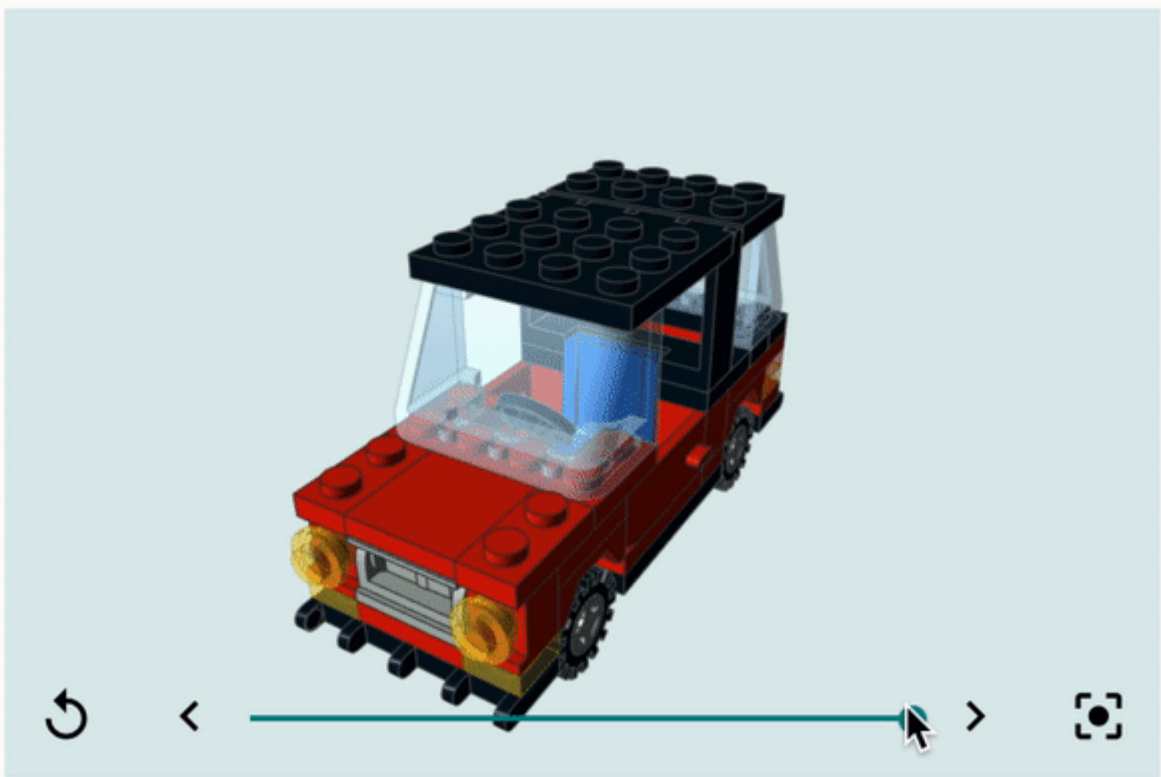
  static styles = css`
    /* ... */
    mwc-slider {
      flex-grow: 1;
    }
  `;

  async firstUpdated() {
    if (this.slider) {
      await this.slider.updateComplete
      this.slider.layout();
    }
  }

  render() {
    return html`
      ${this._renderer.domElement}
      <div id="controls">
        <mwc-icon-button @click=${this._restart} icon="replay"></mwc-icon-button>
        <mwc-icon-button @click=${this._stepBack} icon="navigate_before"></mwc-icon-button>
        <mwc-slider
          step="1"
          pin
          markers
          min="1"
          max=${ifDefined(this._numConstructionSteps)}
          ?disabled=${this._numConstructionSteps === undefined}
          value=${ifDefined(this.step)}
          @input=${(e: CustomEvent) => this.constructionStep = e.detail.value}
        ></mwc-slider>
        <mwc-icon-button @click=${this._stepForward} icon="navigate_next"></mwc-icon-button>
        <mwc-icon-button @click=${this._resetCamera} icon="center_focus_strong"></mwc-icon-
button>
      </div>
    `;
  }
}

```

最終產品如下！



步驟 7 · 約 1 分鐘

7. 結論

我們學到許多關於如何使用 lit-element 建構專屬 HTML 元素的知識。我們學到如何：

- 定義自訂元素
- 宣告屬性 API
- 為自訂元素算繪檢視區塊
- 封裝樣式
- 使用事件和屬性傳遞資料

如要進一步瞭解 lit-element，請參閱[官方網站](#)。

您可以在 stackblitz.com/edit/brick-viewer-complete 查看已完成的 brick-viewer 元素。

brick-viewer 也會透過 NPM 出貨，您可以在這裡查看來源：[Github 存放區](#)。
